Agents for Temporal Grounding

February 11, 2026

Abstract

We study temporal grounding: predicting a distribution over times a text sample could have been produced using public information, to support temporal data bucketing for LLM training. We introduce a multi-agent pipeline that extracts time-anchored entities, grounds them with web evidence, and aggregates conservative year estimates. We introduce an evaluation suite based on human-annotated samples from Tulu-3 to measure the efficiency of the approach and compare against several baselines. We then demonstrate scalability by applying the pipeline to a large subset of the Tulu-3 SFT, RLVR, and DPO datasets.

1 Introduction

Problem setting. We first fix notation. Let x be a text sample and let $t \in \mathbb{R}$ denote time. For each text sample x, our goal is to produce a probability distribution

$$x \mapsto p_x(t),$$

where $p_x(t)$ represents a probability density over times at which x could have been produced using only publicly available knowledge as of time t. We refer to this notion of time as the *epistemic availability time*.

We refer to the task of estimating $p_x(t)$ as *temporal grounding*. In this work, $p_x(t)$ is defined to reflect a downstream use case of interest: labeling and bucketing training data for a language model with a custom knowledge cutoff year τ (e.g., backtesting NLP-based models and what-if scenario analysis under historical knowledge constraints). Operationally, we use LLM-based estimators to approximate $p_x(t)$ (or a year-discretized analogue), which can then be used to produce either a hard year label or a soft inclusion weight for training at cutoff τ.

Desired properties of $p_x(t)$. For our downstream application, we highlight four desired properties that $p_x(t)$ should satisfy.

1. **No temporal leakage.** A model trained with data up to year τ should not encode knowledge of facts that became publicly knowable after τ. Accordingly, $p_x(t)$ should not assign mass to times earlier than τ for text that depends on post-τ knowledge.

2. **Timely availability of explicit events.** If a text references an explicit, time-anchored event that became publicly knowable in year y (e.g., a product launch), then $p_x(t)$ should place all probability mass at that year, ensuring that a model trained through year y can correctly recall that fact.

3. **Historically appropriate implicit knowledge.** Text often reflects implicit social knowledge or public salience rather than discrete events. For example, the statement "There has been a lot of talk about global warming on the news" could plausibly be written across a wide range of years, with increasing plausibility over time. A model trained only through the 1970s should not exhibit the level or framing of concern characteristic of much later discourse; such behavior would indicate temporal leakage. Accordingly, $p_x(t)$ should capture gradual shifts in public understanding rather than sharp thresholds.

4. **Alignment with epistemic availability rather than authorship time.** Explicit cues often imply a single earliest admissible year, while implicit cues naturally induce a distribution over a range of years. These distinctions correspond to different notions of time, including *event time*, *epistemic availability time*, *authorship or publication time*, and *stylistic or discourse time*. For our purposes, epistemic availability time is the most appropriate notion: an event may be known to insiders before becoming public, but a time-bounded model should not rely on insider knowledge. Likewise, authorship time can be misleading, since a contemporary author may deliberately write text consistent with much earlier knowledge and norms. In such cases, the relevant quantity is when the text *could have been written*, rather than when it actually was.

1.1 Examples

The following examples illustrate when a point mass is appropriate and when a distribution over time is more appropriate.

Explicit example 1 (point mass). "The first iPhone was released in 2007." This statement is anchored to a specific public event year, so $p_x(t)$ should concentrate at 2007.

Explicit example 2 (point mass). "Barack Obama was inaugurated for a second term as president last week." This is tied to a single widely known public event, so $p_x(t)$ should be concentrated at 2013.

Implicit example 1 (spread distribution). "There has been a lot of talk about global warming on the news." Public salience grows gradually over time rather than changing at one date, so admissibility is naturally spread across years.

Implicit example 2 (spread distribution). "People are increasingly using smartphones for everyday tasks." Adoption and social visibility increase over multiple years, so a distribution over admissible years is more appropriate than a single year.

2 Method

Entity types. We group time-anchored entities into three categories: explicit, implicit, and timeless, based on how their admissible time is determined under our notion of epistemic availability.

We define *explicit* entities as those with a single earliest admissible year at year-level granularity (e.g., a concrete product launch, public announcement, or release date). For such entities, admissibility uncertainty is negligible, and the entity induces a point-mass distribution over time. For example, "I just bought the first iPhone that was released last week" is explicit: the admissible time concentrates on the release year of the first iPhone, so the entity-level admissible-time distribution is a delta mass at that year.

We define *implicit* entities as those without a single right admissible year, where admissibility depends on diffuse public salience, gradual adoption, or evolving discourse, and may reasonably vary across observers. For example, "There has been a lot of talk about global warming on the

news" is implicit: media coverage and public concern intensified over decades, making it difficult to identify a unique earliest admissible year. As a result, the admissible time for an implicit entity is itself distributed over a range of years.

Finally, *timeless* samples contain no time-anchored entities and impose no temporal constraints.

Entity-level admissible-time representation. For each extracted entity e, we first classify it as explicit or implicit with respect to epistemic availability, and then assign an admissible-time representation:

- For an explicit entity, we assign a single admissible year y_e, corresponding to a degenerate admissible-time distribution $T_e = y_e$.
- For an implicit entity, we assign an admissible-time support interval $[a_e, b_e]$ and, optionally, a discrete probability mass function $p_e(t)$ over years in this interval. The interval defines the support of admissible times, while the probabilities capture uncertainty over when the entity becomes publicly salient.

When probabilities are not explicitly specified, the interval alone is retained (e.g., for conservative aggregation). The representation for implicit entities is intentionally flexible: downstream applications may interpret it as inducing a distribution, while our core method does not assume a particular parametric form.

Operational pipeline. Figure 1 summarizes our end-to-end implementation. We operationalize the method as a multi-agent pipeline with the following steps:

1. **Entity Extraction Agent:** Extract time-anchored entities from the question and answer bundle.
2. **Entity Type Classification Agent:** Classify each extracted entity as explicit or implicit with respect to epistemic availability.
3. **Reasoner Agent:** For each entity, assign an initial admissible-time estimate:
   - explicit entities receive a best-estimate year,
   - implicit entities receive an admissible-time interval and, optionally, per-year probabilities.
4. **Search Query Agent:** Generate standalone search queries for each entity.
5. **Internet Search Agent:** Retrieve external evidence to ground admissible-time estimates.
6. **Reasoner/Synthesizer Agent:** Update entity-level admissible-time representations based on retrieved evidence.

Sample-level aggregation. Let T_e denote the admissible-time random variable induced by entity e. The admissible time of the full sample is defined as

$$T_{\text{sample}} = \max_e T_e,$$

corresponding to the earliest year at which all required knowledge could simultaneously be publicly available. This aggregation rule enforces a strict no-leak constraint: the sample is admissible at year t if and only if every referenced entity is admissible by t.

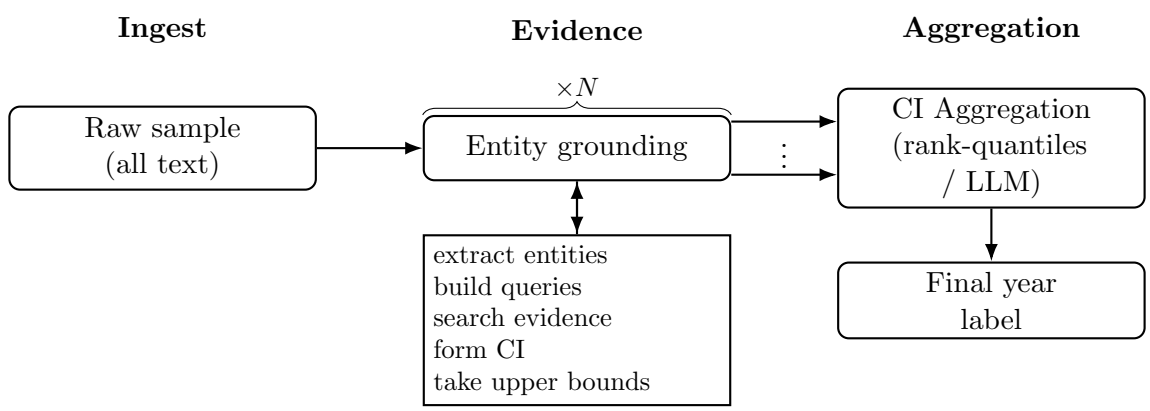


Figure 1: Filtering pipeline for a single sample. Evidence extraction and CI aggregation are repeated for N samples, then merged into a final year label.

Operationally, this definition induces a sample-level admissible-time distribution. In particular, when a sample contains both explicit and implicit entities, probability mass from implicit entities that falls before an explicit entity's admissible year is clipped to that year, resulting in a spike at the explicit year and a tail over later years.

For conservative filtering and labeling, we collapse the sample-level distribution to a single year by taking the maximum upper bound across entity-level admissible-time supports. However, retaining the full distribution enables alternative downstream policies, such as random bucketing, stochastic cutoff simulation, or soft inclusion weighting for year-bounded training.

Evaluation metrics. For now, we evaluate metrics on the *implicit* subset, where gold labels are distributions rather than single years. Let $p_x^{\mathrm{gt}}(t)$ be the gold admissible-time distribution and let $y_x^{\min} = \inf\{t : p_x^{\mathrm{gt}}(t) > 0\}$ be its earliest admissible support year.

Unified primary metric. To evaluate both point and distribution outputs with one score, we use a leakage-penalized Wasserstein objective:

$$\mathrm{Score}_\lambda(x) = W_1\big(q_x, p_x^{\mathrm{gt}}\big) + \lambda \sum_{t < y_x^{\min}} q_x(t),$$

where

$$W_1\big(q_x, p_x^{\mathrm{gt}}\big) = \sum_t \Big| F_{q_x}(t) - F_{p_x^{\mathrm{gt}}}(t) \Big| .$$

The Wasserstein term is interpretable in year units and works uniformly for delta and spread-out distributions; the second term explicitly penalizes leaked mass before the earliest admissible year. We report no-leak accuracy and Score_λ as the primary pair.

Because temporal leakage is asymmetric, we also report a hard leakage indicator:

$$\mathrm{NoLeak}(x) = \mathbf{1}\big[\forall\, t < y_x^{\min},\ q_x(t) = 0\big] .$$

For a single-year prediction $\hat{y}_x$, we treat it as a degenerate distribution $q_x(t) = \mathbf{1}[t = \hat{y}_x]$, so the same definition applies.

3 Related Work

We study a reliability problem that arises in both benchmark construction and downstream deployment: for a natural-language item (question/claim/prompt/answer bundle), what is the *earliest time* at which the item could be produced (or answered) without implicitly using information that only appears later? This section situates our approach at the intersection of (i) methods for tracing *effective* knowledge cutoffs and temporal alignment in LLMs, (ii) time-sensitive QA and temporal reasoning benchmarks, (iii) dynamic evaluation for factual drift/outdatedness, (iv) time-aware modeling (conditioning or routing by time), and (v) leakage/contamination detection and mitigation. While these areas are individually mature, general-purpose *sample-level* dating procedures for arbitrary natural-language items remain comparatively less standardized.

Tracing cutoffs and temporal validity in LLM evaluation. A close conceptual neighbor is Cheng et al. [2024], which defines *resource-level effective cutoffs* and estimates them by probing model perplexity over time-versioned datasets (e.g., monthly Wikipedia snapshots and news buckets). They show that "knowledge cutoffs" are rarely a single clean date: effective cutoffs vary by source and can be blurred by crawl artifacts and near-duplicate content. This motivates item-level temporal controls, since a single global cutoff can be too coarse for evaluation hygiene.

Temporal validity can also break through the retrieval layer. In retrospective settings, practitioners often rely on search-engine date filters to restrict evidence to pre-cutoff time windows. Lahib et al. [2026] audit this practice and show that it can still leak future information even when retrieval appears date-restricted. This supports the need for item-level evidence validation when temporal causality matters.

Temporal generalization analyses broaden the picture: models can be systematically biased across past/present/future contexts. Zhu et al. [2025] propose temporal generalization as an evaluation axis and provide evidence of temporal biases and performance decay. Their perspective reinforces that time-indexed evaluation suites should be treated as a first-class design choice rather than an afterthought.

Time-aware modeling. Beyond evaluation, several works treat time as an explicit modeling variable. Dhingra et al. [2022] interpret language models as temporal knowledge bases and study time-varying facts, proposing timestamp conditioning to better represent and retrieve temporally scoped knowledge. Such mechanisms reduce ambiguity about "what was true when" and provide a natural sink for timestamped supervision derived from curated data.

A complementary architectural approach is temporal routing. Faro et al. [2025] train experts on disjoint temporal slices and mask experts whose training windows end after the query time, enforcing a strict "no future knowledge" constraint at inference time. Relative to our approach, which focuses on assigning time constraints to arbitrary items and using them to filter/stratify data, time-routed models directly encode temporal constraints in the model architecture. Together, they offer a path to both (i) temporally consistent data and (ii) temporally constrained inference.

Time-sensitive QA and temporal reasoning benchmarks. Benchmark families study temporality from different angles: time-sensitive *facts* (answers change over time), temporal *context* (answers depend on when/where asked), and temporal *reasoning* (ordering and temporal relations in text). TimeQA focuses on time-sensitive factual questions grounded in evolving Wikidata facts and aligned evidence sources [Chen et al., 2021]. SituatedQA analyzes questions whose answers depend on temporal and geographic context, showing that systems can fail when context is implicit or

mismatched [Zhang and Choi, 2021]. TORQUE targets temporal reasoning about event ordering in passages (before/after/during) via a reading-comprehension format [Ning et al., 2020]. Collectively, these datasets motivate making temporal scope explicit, since evaluation can be confounded when the temporal frame is implicit or inconsistent.

A core difficulty in time-sensitive QA is disentangling genuine temporal reasoning from memorized factual recall. UnSeenTimeQA addresses this by constructing contamination-resistant scenarios intended to reduce web-searchable memorization [Uddin et al., 2025]. TDBench proposes a systematic evaluation of factual time-sensitive QA using temporal databases [authors, 2026]. In contrast to work that assumes timestamps are known by construction (e.g., from structured sources), our setting targets arbitrary natural-language items whose temporal requirements must be inferred from content and evidence.

Dynamic evaluation and outdatedness detection. Another stream evaluates time-sensitive knowledge by validating against continuously updated structured sources. DyKnow dynamically checks whether LLM outputs match current Wikidata values and measures outdatedness and inconsistency under prompt perturbations [Mousavi et al., 2024]. Dynamic benchmarking work similarly generates time-bucketed test sets from Wikidata and evaluates robustness to factual drift under multiple evaluation "views" [Margatina et al., 2023]. These methods are well-suited to entity-centric facts with structured ground truth; evidence-grounded dating aims to generalize the same instinct to open-ended claims and questions that do not map cleanly onto a knowledge graph.

Contamination, leakage, and decontamination methods. Temporal grounding is also motivated by the contamination literature, which shows that benchmark scores can be inflated by overlap between training and evaluation data. Golchin and Surdeanu [2024] present black-box strategies for tracing contamination, while Golchin and Surdeanu [2025] introduce a quiz-based tool to detect and estimate contamination without training-data access. Li and Flanigan [2024] highlight task-level contamination effects by showing that models often perform markedly better on datasets released before their pretraining collection windows than on datasets released after, complicating "few-shot" claims.

Mitigation approaches include rewriting and filtering evaluation items. CLEAN–EVAL creates cleaner benchmark variants via transformations (e.g., paraphrasing/translation) and semantic filtering to reduce memorization-based score inflation [Zhu et al., 2024b]. Inference-Time Decontamination (ITD) combines detection with rewriting during evaluation to "revive" leaked benchmarks while preserving difficulty [Zhu et al., 2024a]. These strategies are largely orthogonal to temporal dating: rewriting targets surface-form memorization, whereas dating targets causal validity and future-only information. In practice, they can be combined: a time-dated benchmark can also be rewritten to stress-test generalization.

Finally, pretraining-data detection methods offer another complementary tool family. Shi et al. [2024] introduce benchmarks like WIKIMIA and the Min-K% Prob detector to infer whether text was included in pretraining from black-box probabilities. While temporal dating aims to infer the *earliest* plausible time a text could be written, pretraining-data detection aims to infer whether a model has *seen* the text; together, they help diagnose whether strong performance stems from reasoning under appropriate temporal constraints or from memorization/leakage.

Synthesis and remaining gaps. Across these strands, a consistent message emerges: effective cutoffs vary by resource [Cheng et al., 2024], date-filtered retrieval can leak [Lahib et al., 2026], temporal biases exist [Zhu et al., 2025], and contamination can systematically inflate scores [Li and

Flanigan, 2024]. What remains less standardized is a general-purpose, sample-level mechanism that assigns an earliest admissible time to arbitrary natural-language items using evidence and reasoning, and that is convenient enough to serve as a dataset filtering primitive and evaluation control.

4 Evaluation

Annotation protocol. For each sample, the annotator read the question and answer, extracted relevant entities (omitting entities that are clearly older than the dominant ones), and searched to find the concept date for each entity. For each entity, we include a source link to the article stating the concept date unless the fact is trivially known. The full dev-set annotation pass took about 2 hours, or roughly 3.5 minutes per question.

Dev set. We build a 35-question dev set sampled from Tulu-3 and labeled by human annotators to design and iterate on the prompting scheme, entity grounding, and aggregation rules. We use the dev set to refine the prompting scheme and aggregation rules; test-set results are reported separately once all plots are finalized.

Test set. We prepared a held-out test set of 141 samples from Tulu-3. The test set is constructed and annotated in the same way as the dev set, with human annotators providing gold years and source links.

5 Results

Key plots for this analysis are Figures 2 and 3. Figure 2 shows that search grounding improves no-leak accuracy by roughly 10 points across models, with Gemini models reaching 100% no-leak accuracy on the dev set. Figure 3 shows the distribution of predicted year minus gold year, where the most common failure is entity extraction and search grounding appears reliable. We categorize error types using an LLM with access to the ground-truth answer. Figure 4 shows the search-aided prediction error (predicted year minus gold year) by model on the test set, with negative deltas colored by failure type. In many cases the difference reflects the LLM finding later references than the human annotator; this should improve once multiple annotators review each example. Without gold labels, we estimate model conservatism by counting how often each model provides the maximum year for the same prompt.

Other performance metrics. Possible options include an asymmetric error loss averaged over samples, $L = \max(0, -e) + \beta \cdot \max(0, e)$ where $e = \hat{y} - y$, $\hat{y}$ is the predicted year, and y is the gold year. This penalizes underestimates more than overestimates when $\beta < 1$, but it is harder to interpret and requires choosing β. Another option is an equally weighted average of exact year match and no-leak accuracy, which is easy to interpret but does not distinguish between off-by-one errors and large errors.

6 Discussion

Two observations stand out: Gemini 3 Flash performs better than Gemini 3 Pro, and GPT-5-mini is similar to GPT-5.2. It would be valuable to explore whether this stems from smaller models producing more conservative intervals and from the asymmetric risk profile that penalizes underestimates more than overestimates.

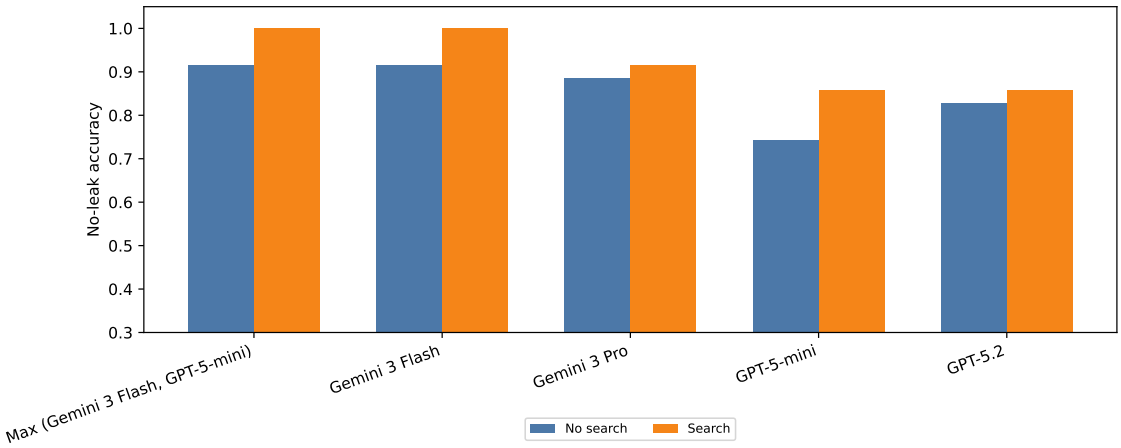


Figure 2: No-leak accuracy (predicted year $\geq$ gold) before vs. after search grounding on the dev set ($N = 35$). Values are proportions without uncertainty bars because each point aggregates the full dev set.

7 Data Filtering

We label each supervised (SFT), preference, and RLVR sample with the minimum calendar year consistent with its question–answer bundle. The prompt structure is detailed in **??**. We considered deterministic filtering, but it was difficult to capture all edge cases with a rule-based approach. The latest sweep (session `2026-01-06_14-10PT`) processed 30,549 SFT examples, 29,510 preference examples, and three RLVR datasets (7,358 GSM, 7,372 MATH, 14,958 IFEval) with a conservative policy that uses the most recent referenced year. Figures 5 to 7 show year and category distributions for each dataset family and validate cutoff integrity.

References

Anonymous authors. Harnessing temporal databases for systematic evaluation of factual time-sensitive question-answering in llms. In *Under review (ICLR 2026 submission on OpenReview)*, 2026. URL https://openreview.net/forum?id=W7RNxsTKKZ.

Wenhu Chen, Xinyi Wang, and William Yang Wang. A dataset for answering time-sensitive questions. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/1f0e3dad99908345f7439f8ffabdffc4-Abstract-round2.html, 2021. NeurIPS 2021 Datasets and Benchmarks.

Jeffrey Cheng, Marc Marone, Orion Weller, Dawn Lawrie, Daniel Khashabi, and Benjamin Van Durme. Dated data: Tracing knowledge cutoffs in large language models, 2024. URL https://arxiv.org/abs/2403.12958.

Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 2022. URL https://aclanthology.org/2022.tacl-1.15/.

Robin Faro, Dongyang Fan, Tamar Alphaidze, and Martin Jaggi. Timoe: Time-aware mixture of language experts, 2025. URL https://arxiv.org/abs/2508.08827.

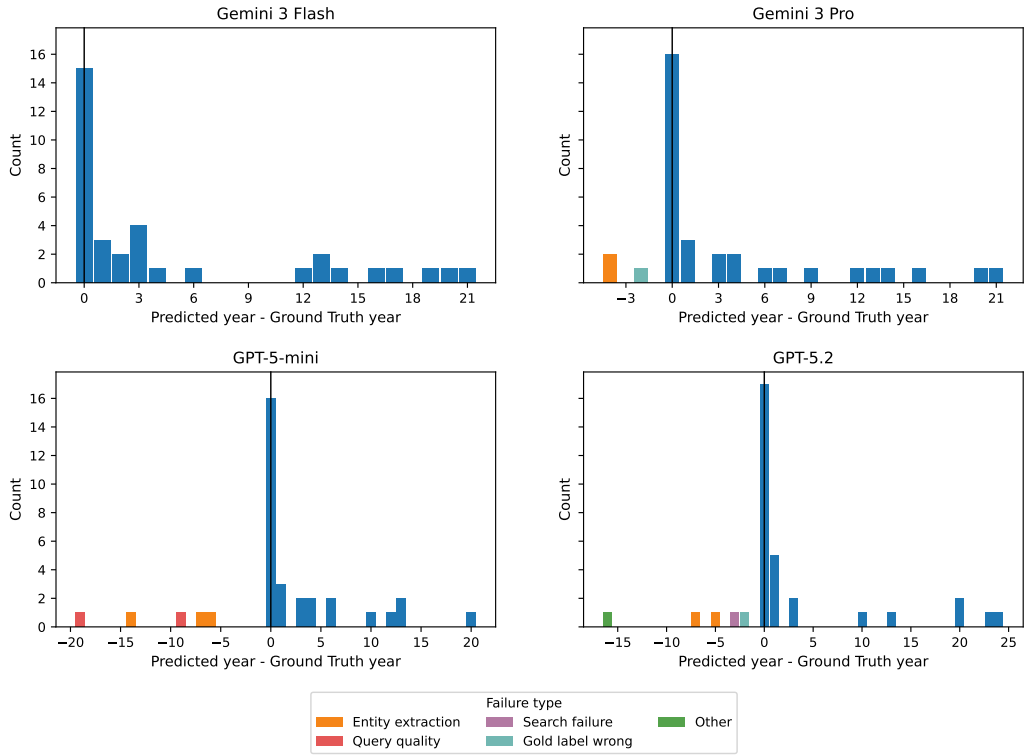


Figure 3: Search-aided prediction error (predicted year minus gold year) by model on the dev set ($N = 35$), with negative deltas colored by failure type. Panels correspond to models.

Shahriar Golchin and Mihai Surdeanu. Time travel in llms: Tracing data contamination in large language models, 2024. URL https://arxiv.org/abs/2308.08493.

Shahriar Golchin and Mihai Surdeanu. Data contamination quiz: A tool to detect and estimate contamination in large language models, 2025. URL https://arxiv.org/abs/2311.06233.

Ali El Lahib, Ying-Jieh Xia, Zehan Li, Yuxuan Wang, and Xinyu Pi. Temporal leakage in search-engine date-filtered web retrieval: A case study from retrospective forecasting, 2026. URL https://www.arxiv.org/abs/2602.00758.

Changmao Li and Jeffrey Flanigan. Task contamination: Language models may not be few-shot anymore, 2024. URL https://arxiv.org/abs/2312.16337.

Katerina Margatina, Shuai Wang, Yogarshi Vyas, Neha Anna John, Yassine Benajiba, and Miguel Ballesteros. Dynamic benchmarking of masked language models on temporal concept drift with multiple views, 2023. URL https://arxiv.org/abs/2302.12297.

Seyed Mahed Mousavi, Simone Alghisi, and Giuseppe Riccardi. Dyknow: Dynamically verifying time-sensitive factual knowledge in llms, 2024. URL https://arxiv.org/abs/2404.08700.

Qiang Ning, Hao Wu, Rujun Han, Nanyun Peng, Matt Gardner, and Dan Roth. Torque: A reading comprehension dataset of temporal ordering questions. In *EMNLP 2020*, 2020. URL https://aclanthology.org/2020.emnlp-main.88/.

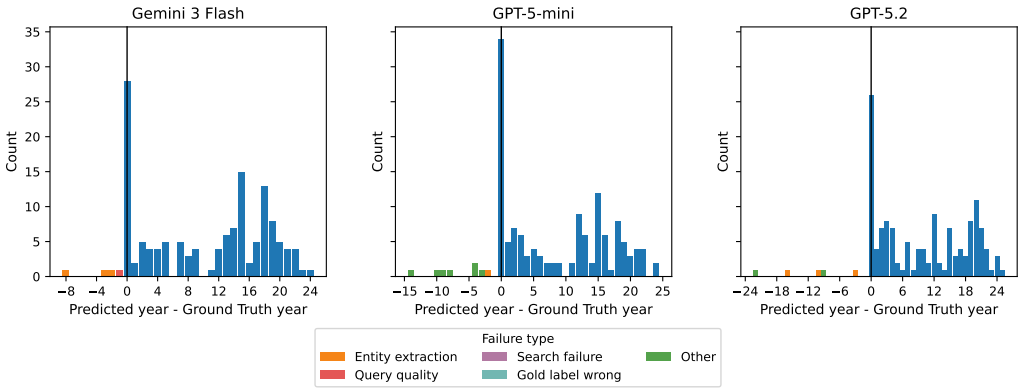
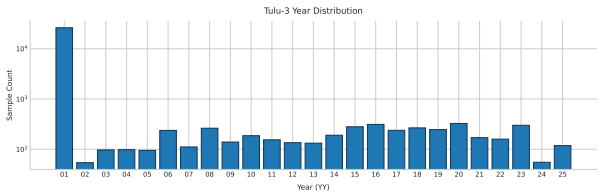
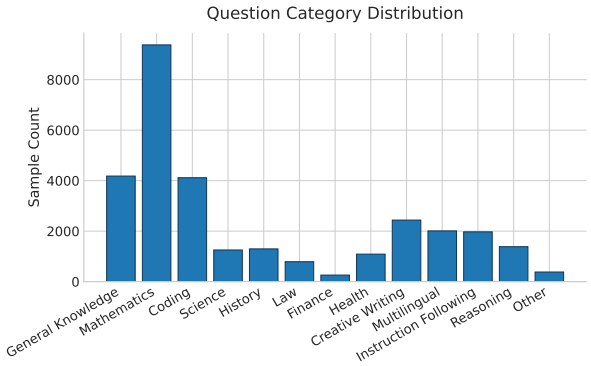


Figure 4: Search-aided prediction error (predicted year minus gold year) by model on the test set ($N = 141$), with negative deltas colored by failure type. Panels correspond to models.



(a) Year distribution (SFT).



(b) Category distribution (SFT).

Figure 5: Filtering summary for the TÜLU-3 SFT mixture (session `2026-01-06_14-10PT`, $n = 30{,}549$). Histograms are computed from the filtered sample set.

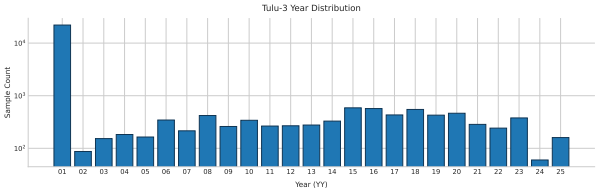
Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models, 2024. URL https://arxiv.org/abs/2310.16789.

Md Nayem Uddin, Amir Saeidi, Divij Handa, Agastya Seth, Tran Cao Son, Eduardo Blanco, Steven R. Corman, and Chitta Baral. Unseentimeqa: Time-sensitive question-answering beyond llms' memorization. In *ACL 2025*, 2025. URL https://aclanthology.org/2025.acl-long.94/.

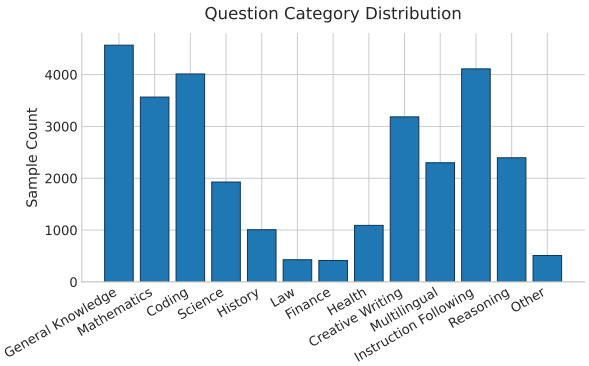
Michael J.Q. Zhang and Eunsol Choi. Situatedqa: Incorporating extra-linguistic contexts into qa. In *EMNLP 2021*, 2021. URL https://aclanthology.org/2021.emnlp-main.586/.

Chenghao Zhu, Nuo Chen, Yufei Gao, Yunyi Zhang, Prayag Tiwari, and Benyou Wang. Is your llm outdated? a deep look at temporal generalization. In *NAACL 2025 (Long Papers)*, 2025. URL https://aclanthology.org/2025.naacl-long.381/.

Qin Zhu, Qingyuan Cheng, Runyu Peng, Xiaonan Li, Tengxiao Liu, Ru Peng, Xipeng Qiu, and Xuanjing Huang. Inference-time decontamination: Reusing leaked benchmarks for large language



(a) Year distribution (DPO).

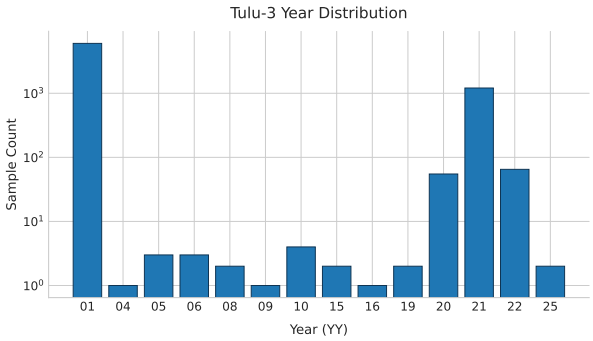


(b) Category distribution (DPO).

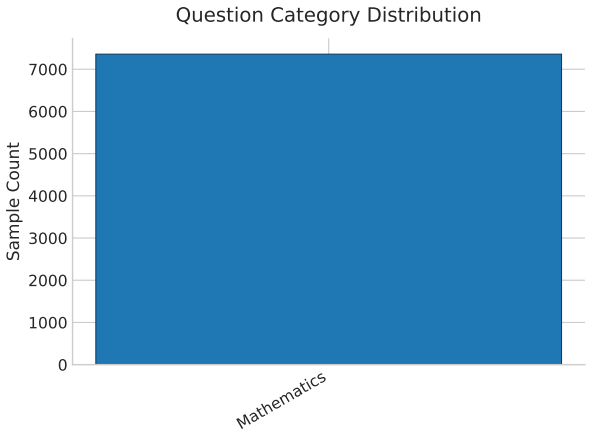
Figure 6: Filtering summary for the TÜLU-3 preference mixture (session `2026-01-06_14-10PT`, $n = 29{,}510$). Histograms are computed from the filtered sample set.

model evaluation. In *Findings of EMNLP 2024*, 2024a. URL https://aclanthology.org/2024.findings-emnlp.532/.

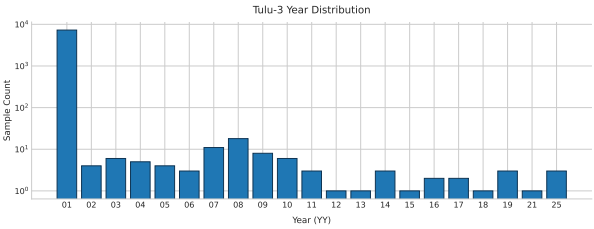
Wenhong Zhu, Hongkun Hao, Zhiwei He, Yunze Song, Yumeng Zhang, Hanxu Hu, Yiran Wei, Rui Wang, and Hongyuan Lu. Clean–eval: Clean evaluation on contaminated large language models. In *Findings of NAACL 2024*, 2024b. URL https://aclanthology.org/2024.findings-naacl.53/.



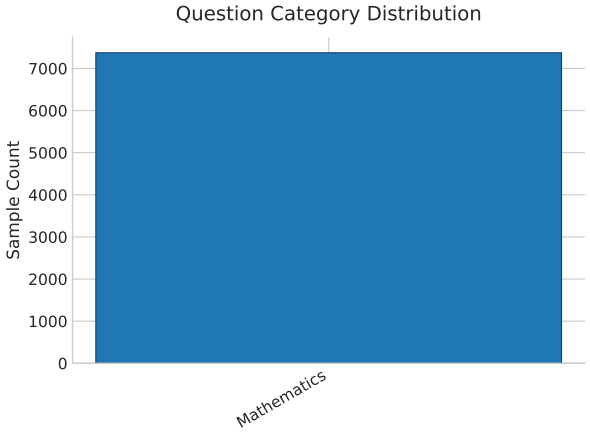
(a) Year distribution (RLVR-GSM).



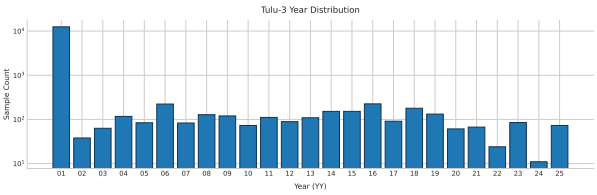
(b) Category distribution (RLVR-GSM).



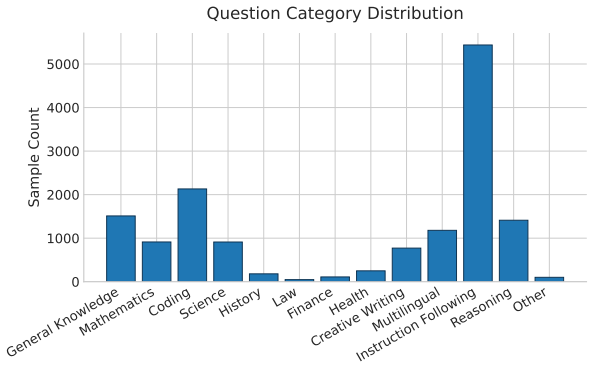
(c) Year distribution (RLVR-MATH).



(d) Category distribution (RLVR-MATH).



(e) Year distribution (RLVR-IFeval).



(f) Category distribution (RLVR-IFeval).

Figure 7: Filtering summary for the RLVR datasets (session `2026-01-06_14-10PT`, GSM $n = 7{,}358$, MATH $n = 7{,}372$, IFEval $n = 14{,}958$). Histograms are computed from the filtered sample sets.